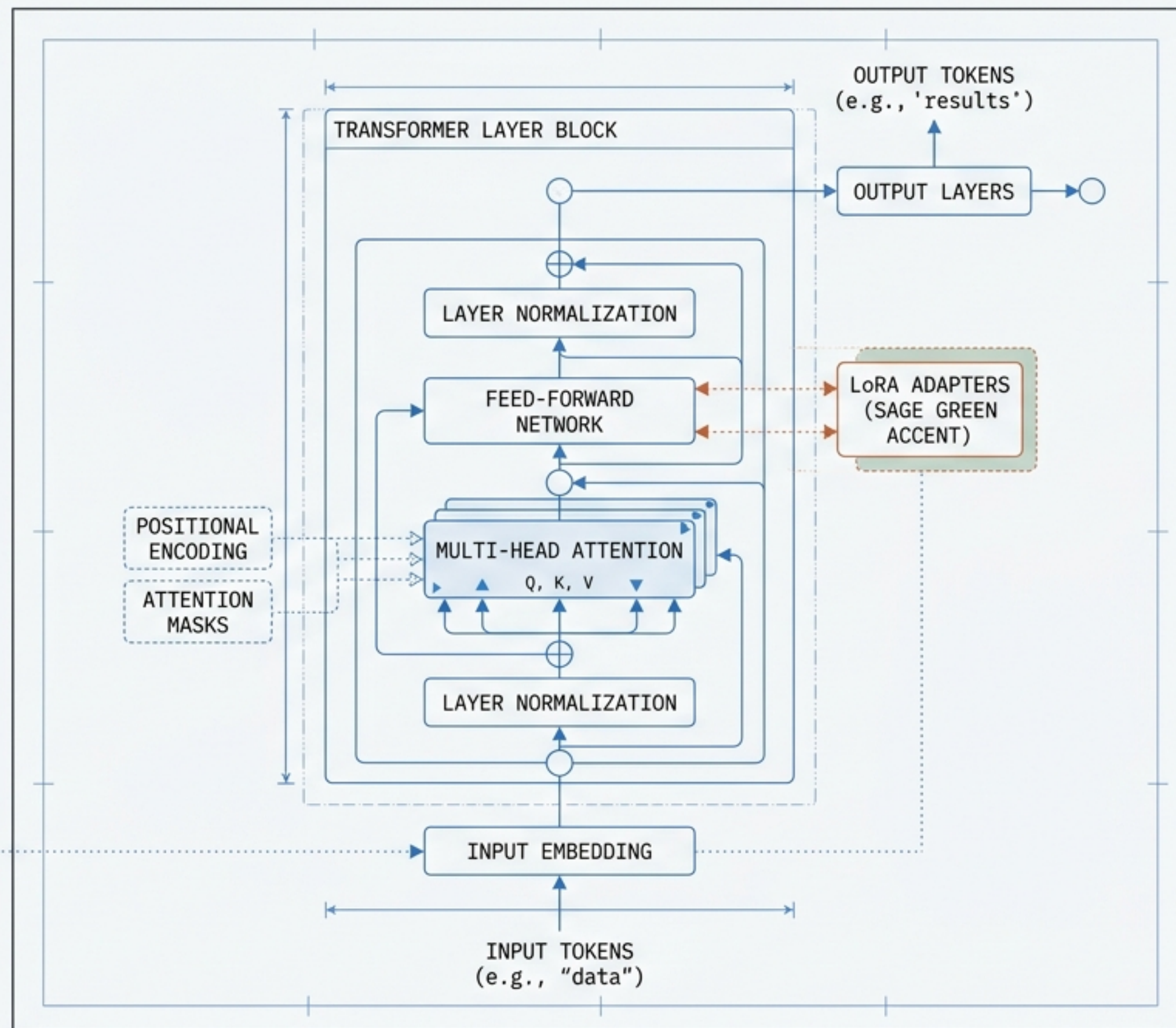


De Usar Modelos a Diseñar Sistemas

Fundamentos arquitectónicos para la IA en producción:

Transformers,
Tokenization y LoRA.



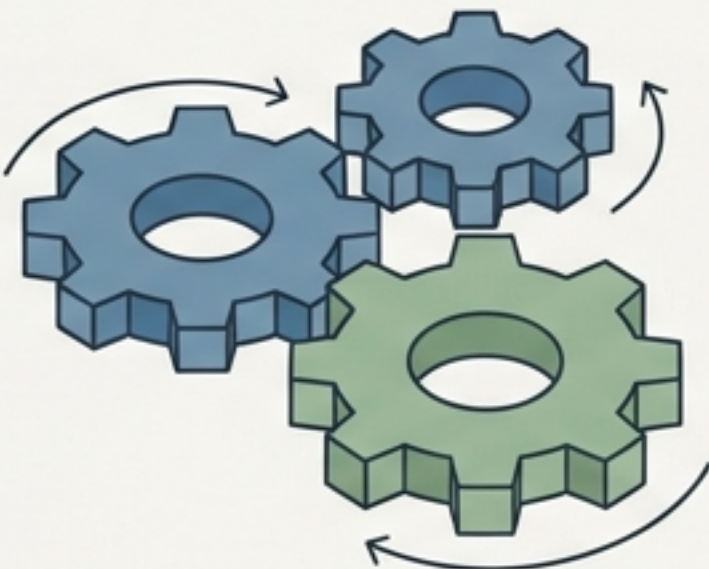
La caja negra no escala en producción

El Enfoque Superficial



Desplegar sin entender la arquitectura genera estrés en la infraestructura. Consecuencias: **Latencia impredecible**, sobrecostos por **ciclo de CPU** y fallos en contextos nuevos (ej. **chatbots financieros estrictos**).

El Enfoque de Ingeniería



Dominar los mecanismos internos permite alinear **requisitos técnicos** con prioridades del negocio.

Los 3 Pilares del Diseño:

1. Arquitectura
(Atención)

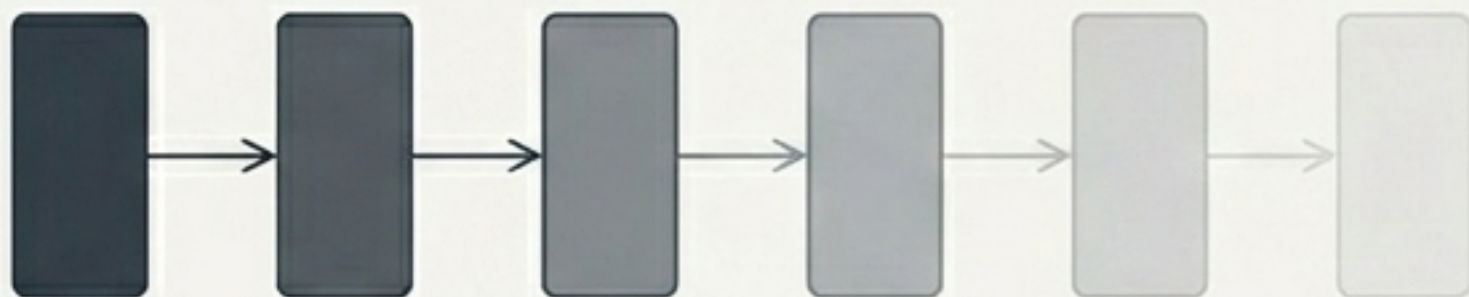
2. Preprocesamiento
(Tokenización)

3. Adaptación
(Eficiencia)

Cada **milisegundo** se traduce en costos concretos.

Rompiendo el cuello de botella secuencial

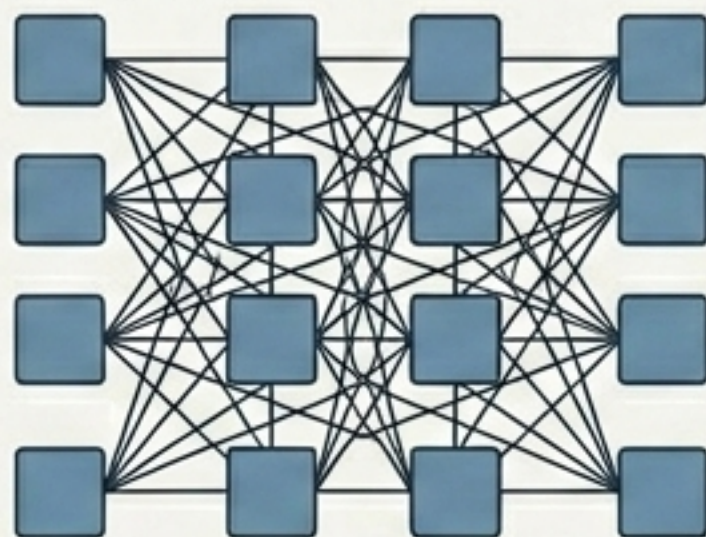
El Pasado



Modelos Secuenciales (RNN/LSTM)

Las redes leen paso a paso. Cuanto más larga es la secuencia, más información se pierde. Es imposible paralelizar eficientemente el cómputo.

El Paradigma Transformer



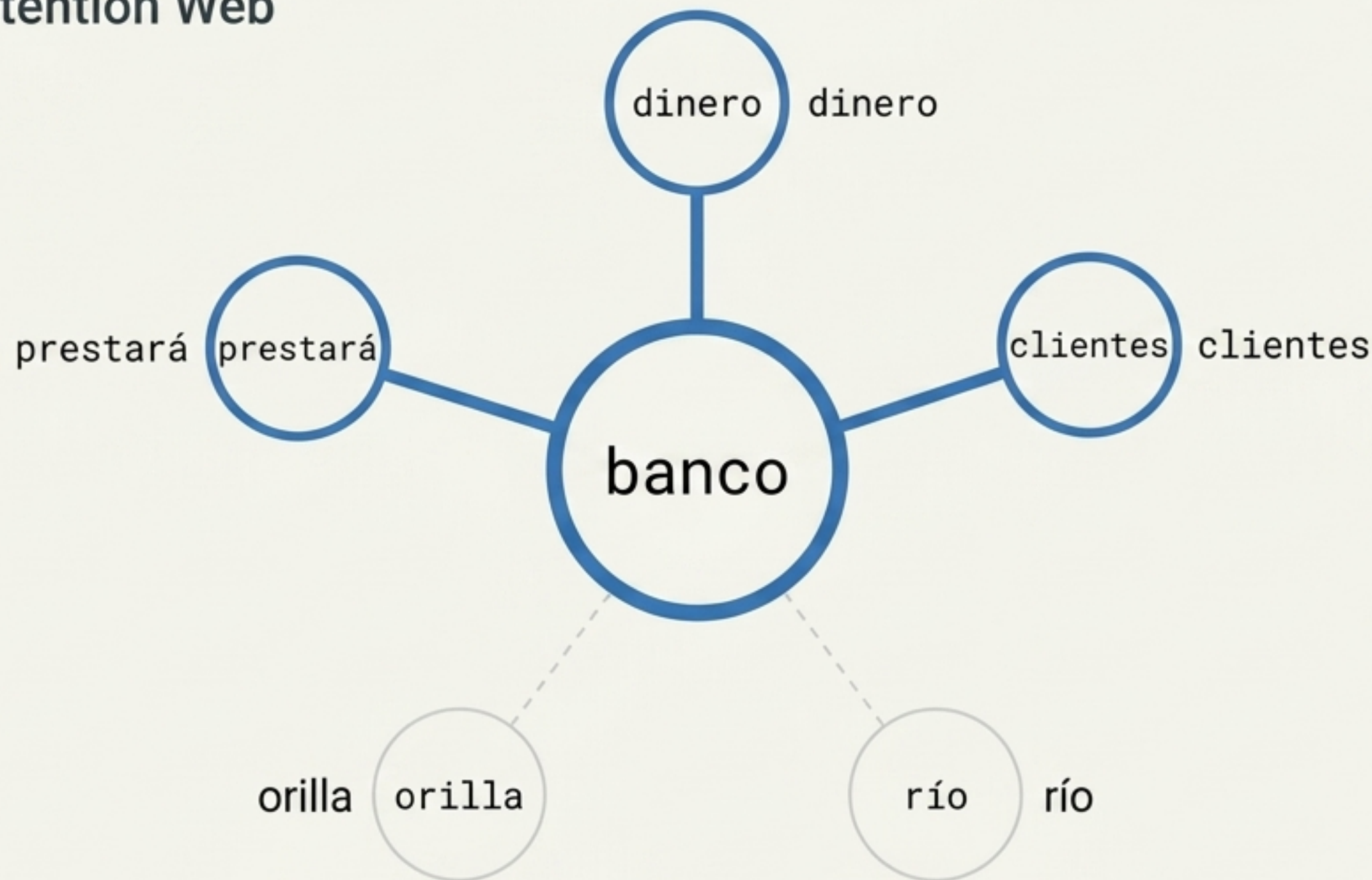
Procesamiento Global

Analiza todo simultáneamente. Vital para capturar dependencias de largo alcance (ej. conectar una cláusula en el primer párrafo de un contrato legal con una afirmación en la última página).

Self-Attention: El contexto lo es todo

¿Cómo sabe el modelo si la palabra “banco” es una institución o la orilla de un río?

Attention Web

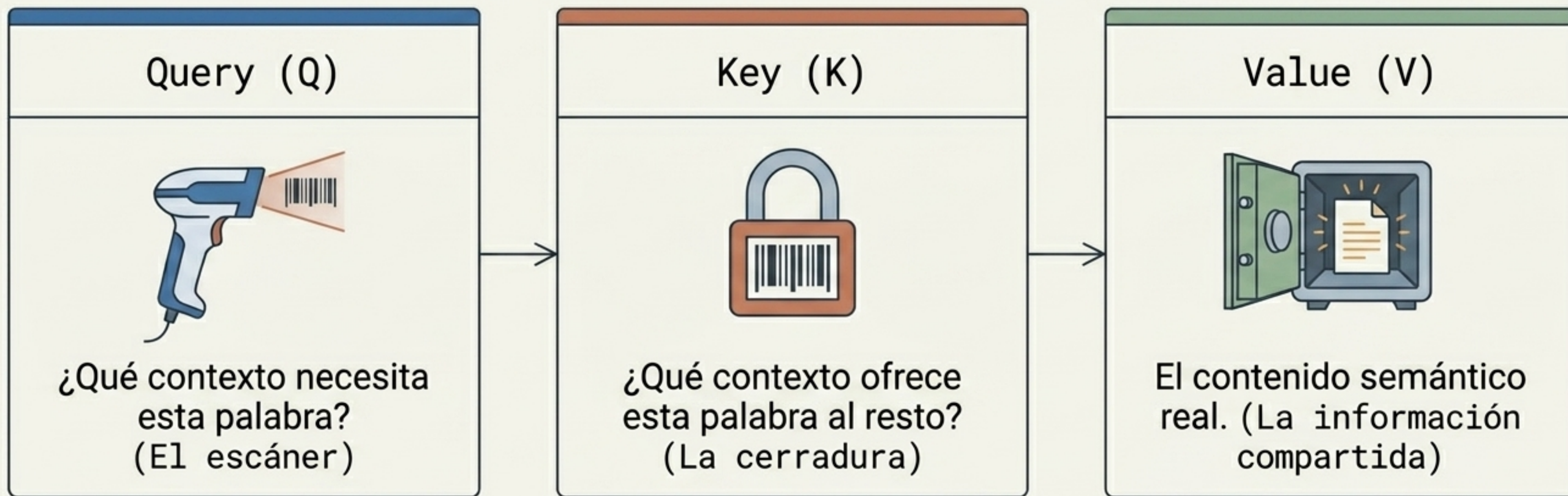


El mecanismo de self-attention permite que el modelo mire **toda la oración a la vez** y calcule un **mapa ponderado**.

Asigna **puntajes de importancia** a todas las demás palabras, **relevantes** son para **entender** la palabra actual, sin importar su distancia en el texto.

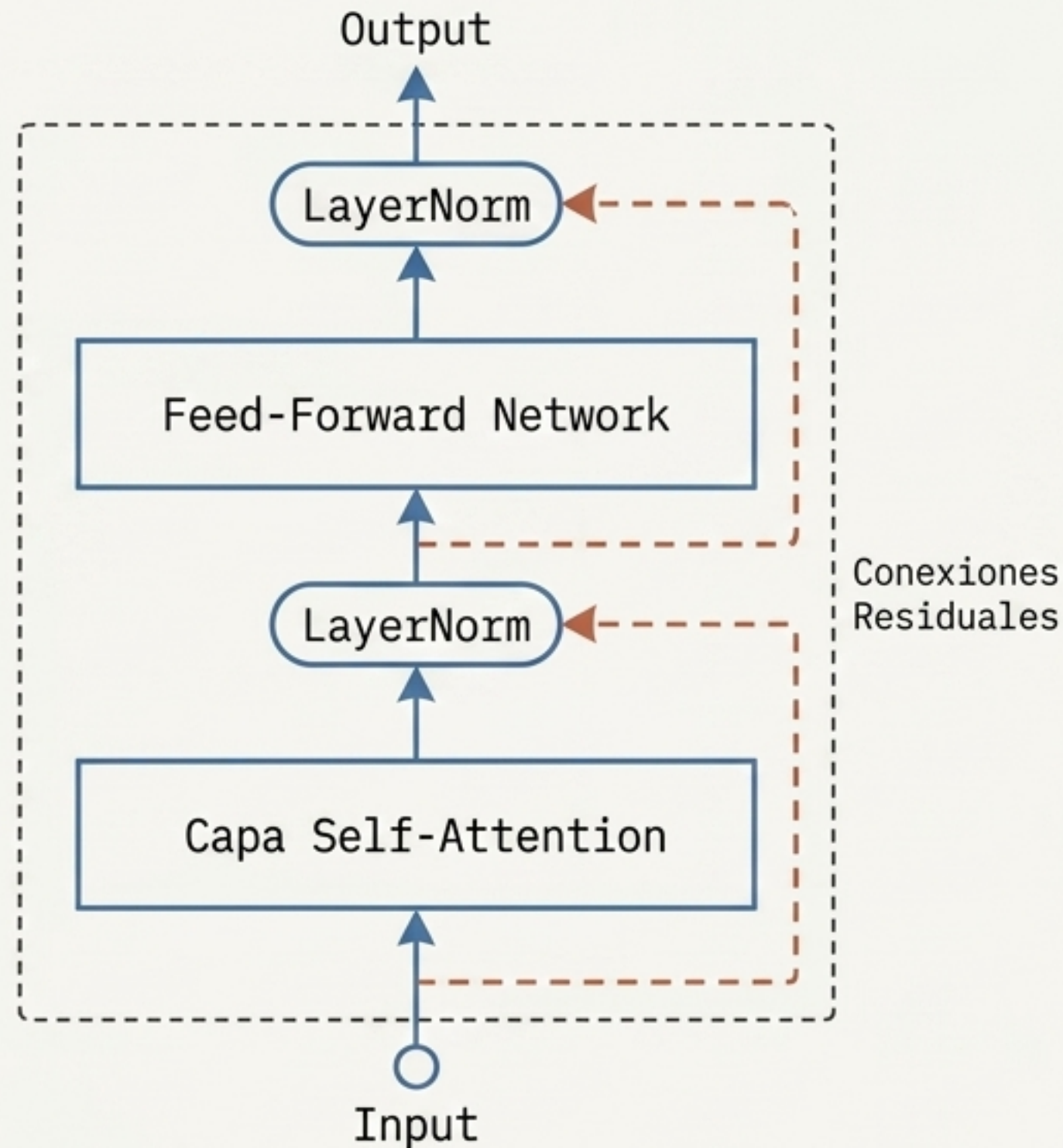
La mecánica de la atención: Query, Key, Value

Cada embedding se transforma en tres versiones vectoriales para calcular la atención:



La coincidencia matemática (multiplicación) entre Q y K genera el puntaje de atención, el cual determina cuánto del V se suma al resultado final.

El Bloque Transformer: La unidad de procesamiento



1. Capa Self-Attention

Reúne información contextual de toda la secuencia.

2. Feed-Forward Network (FFN)

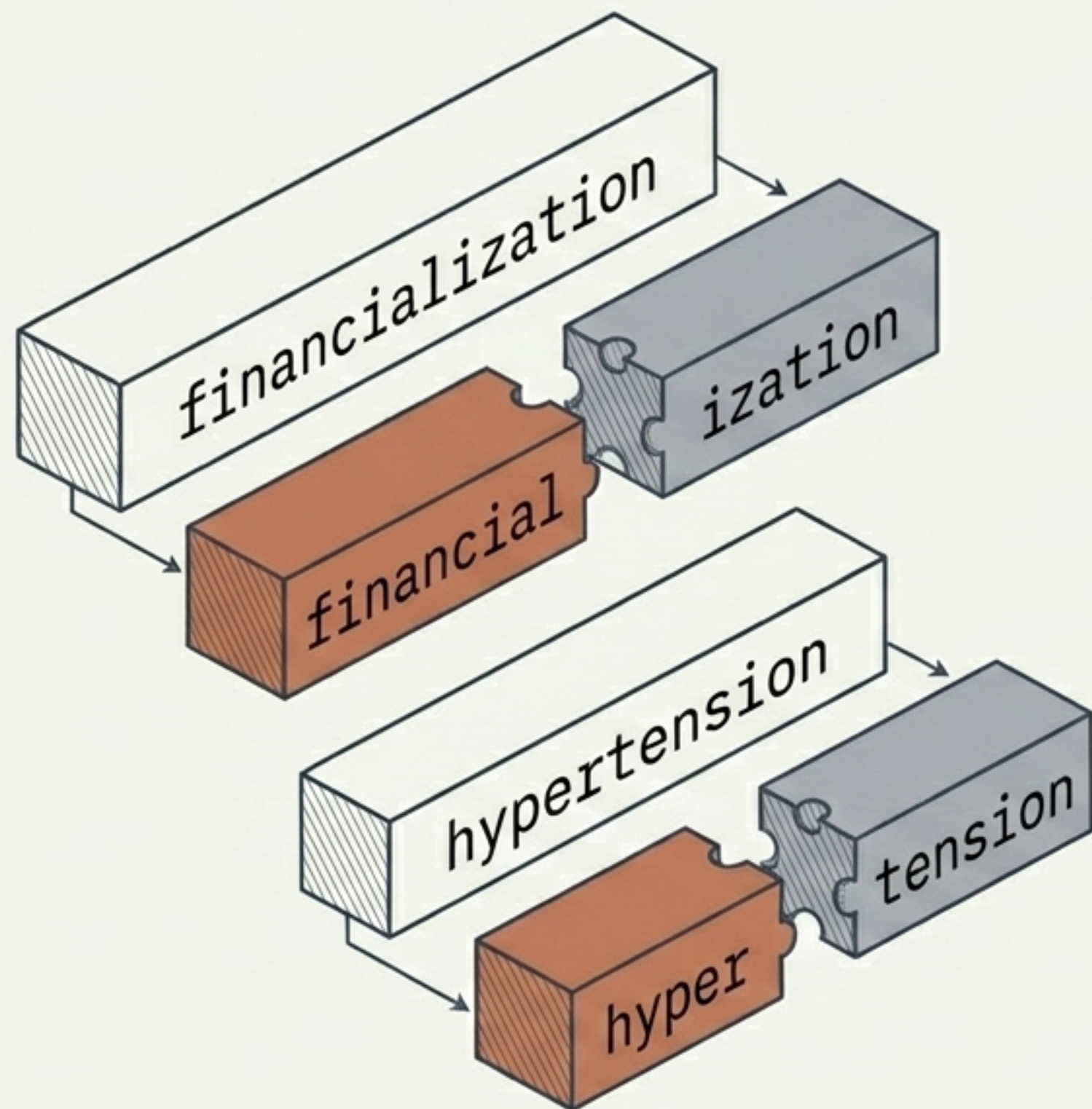
Un perceptrón independiente por posición que refina los vectores contextualizados.

3. LayerNorm y Residuales

Estabilizan el aprendizaje y evitan la pérdida de información (*catastrophic forgetting*) al saltar capas.

Los modelos modernos apilan decenas de estos bloques para capturar relaciones jerárquicas complejas.

Tokenization: El puente entre texto y cómputo



El Problema del Vocabulario Fijo

Los vocabularios fijos se rompen ante neologismos, errores ortográficos o jerga. Sustituir palabras desconocidas por tokens genéricos [UNK] destruye la información.



La Solución: Subwords

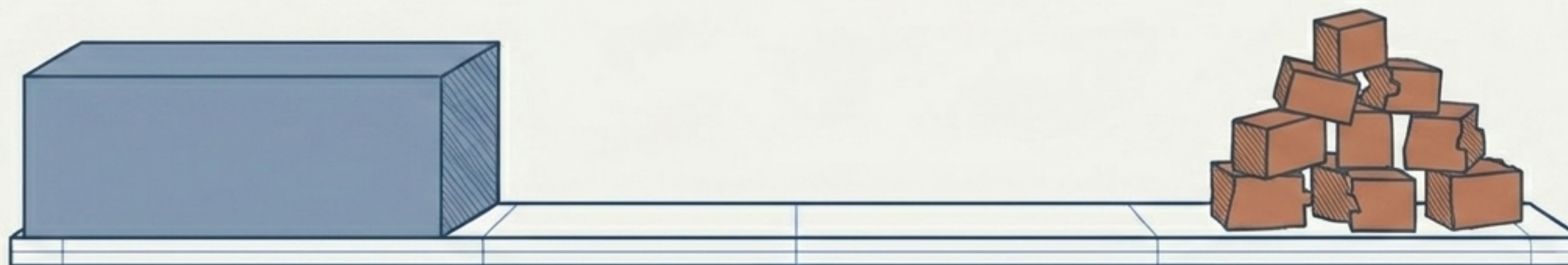
En lugar de palabras completas, el texto se divide en piezas reutilizables. Esto permite a los modelos inferir el significado de palabras que nunca han visto basándose en sus raíces conocidas.

Evaluando estrategias: BPE vs. WordPiece

Dimensión	Byte Pair Encoding (BPE)	WordPiece
Objetivo Principal	Combinar el par más frecuente	Maximizar la verosimilitud del modelo
Regla de Construcción	Greedy, busca coincidencias largas	Minimiza tokens usando merges entrenados
Manejo de Desconocidos	Robusto vía subwords	Usa prefijos ## para distinguir continuaciones

Ejemplos de Procesamiento	
Input: cat -> BPE: cat WP: cat	Input: hypertension -> BPE: hyper, tension WP: hyper, ##tension

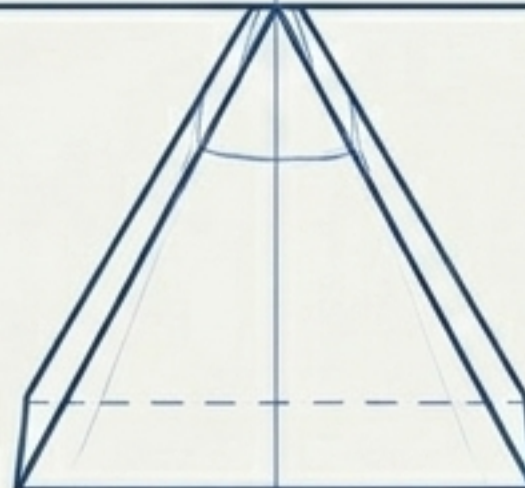
El trade-off de la Tokenization



Vocabularios Grandes

Impacto: Menos *tokens* por input. Secuencias de texto más cortas.

Costo: Tablas de *embeddings* gigantes. Mayor consumo crudo de memoria RAM y alto costo de almacenamiento.



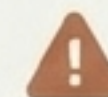
Eficiencia
de Sistema

Vocabularios Pequeños



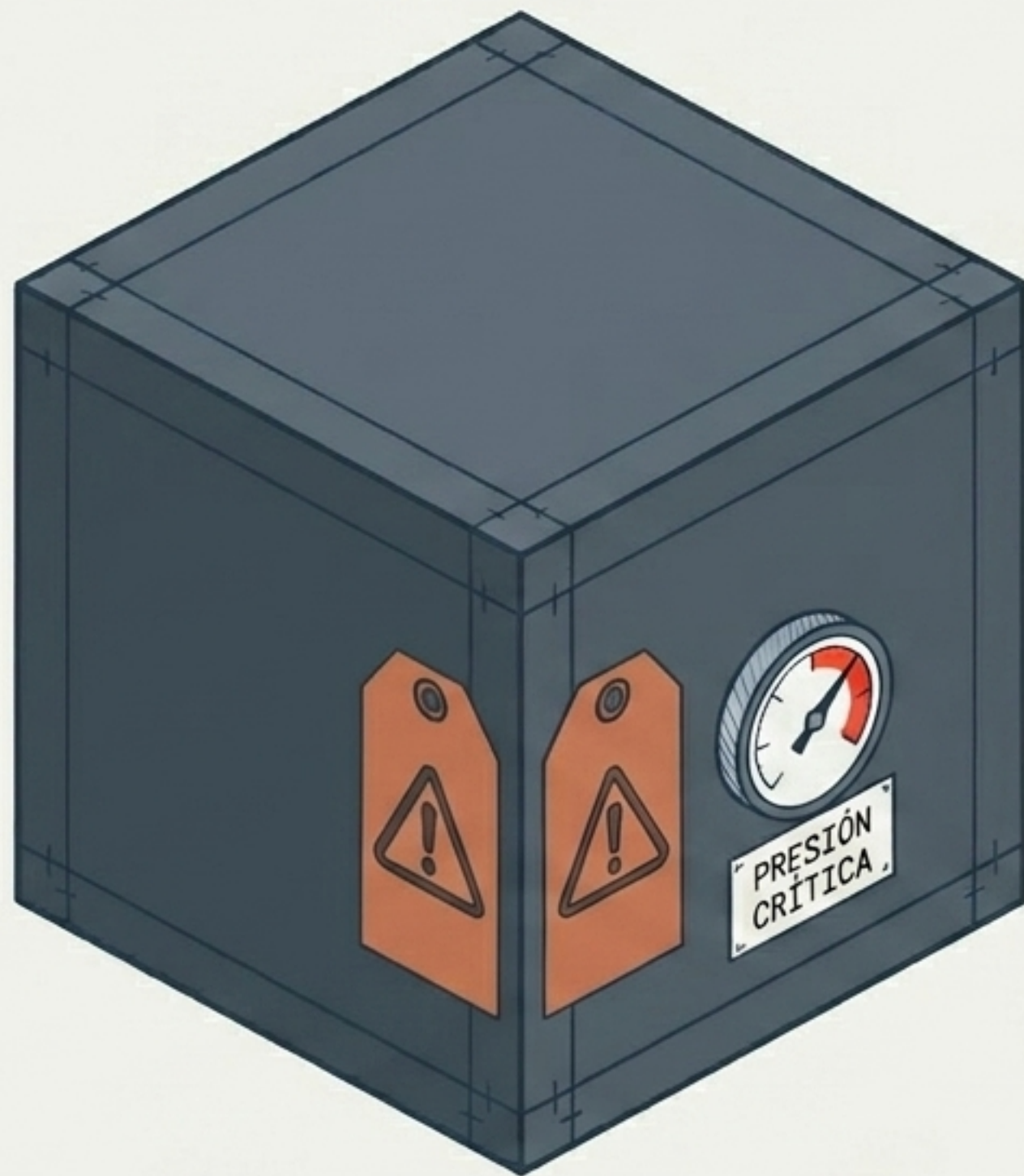
Impacto: Ideal para infraestructuras con restricciones severas de memoria.

Costo: Mayor fragmentación (más *tokens* por palabra). Dispara el costo de cómputo en la capa de atención y aumenta la latencia.



Adaptar el **modelo** requiere alinear el tamaño del **vocabulario** con las limitaciones físicas del **hardware** de despliegue.

El problema del Full Fine-Tuning



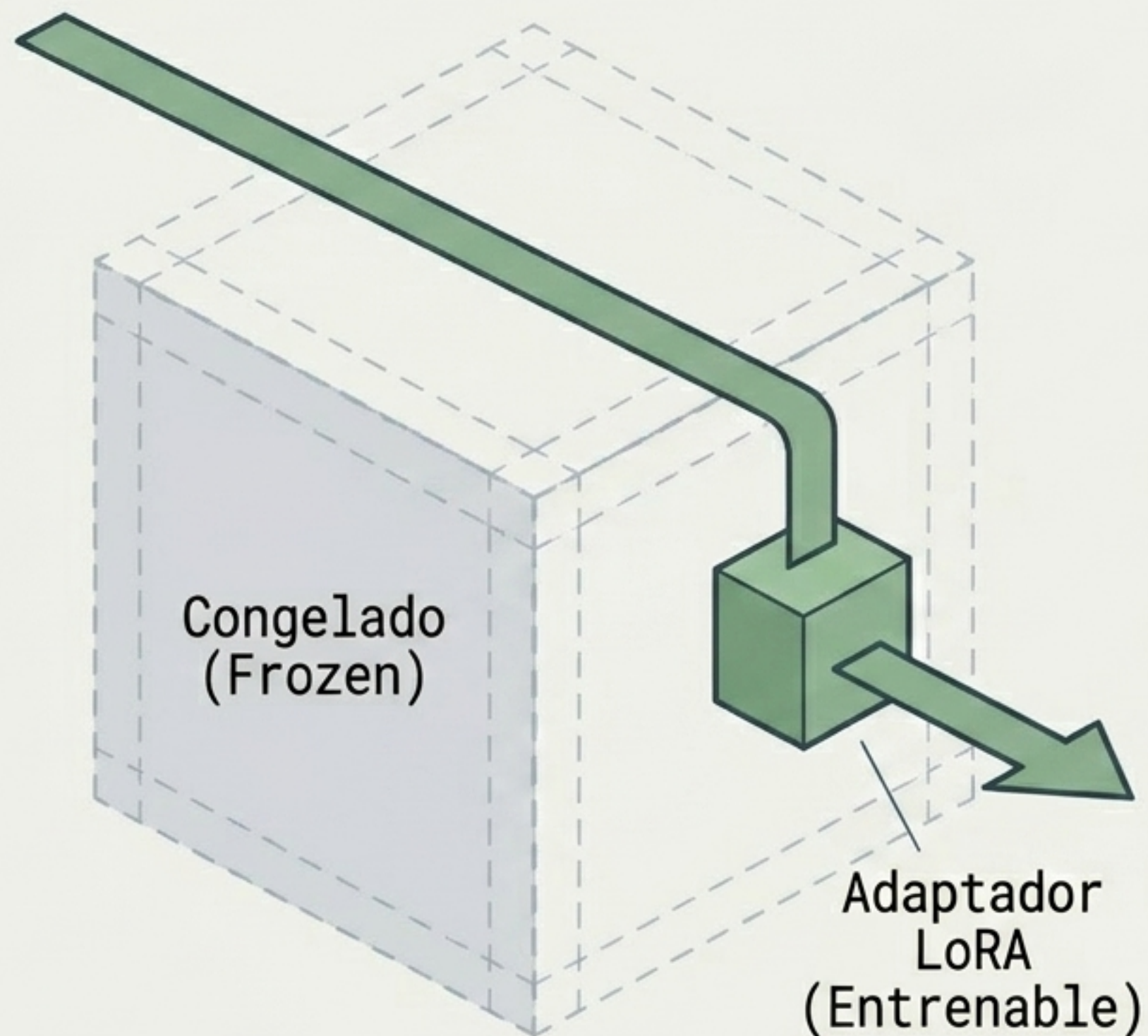
Contexto

Escenario: Startup médica licencia un modelo general. En conversaciones casuales funciona impecable, pero falla gravemente en diagnósticos especializados. Requiere precisión clínica urgente.

La Realidad del 100%

- ☒ Semanas de esfuerzo de ingeniería y clústeres masivos de GPUs.
- ☒ Riesgo crítico de desestabilizar comportamientos útiles previos.
- ☒ Almacenamiento de variantes gigantescas del modelo para cada cliente.

PEFT y LoRA: Adaptación Quirúrgica



Concepto PEFT

Parameter-Efficient Fine-Tuning congela la base matemática del modelo original y añade módulos minúsculos entrenables.

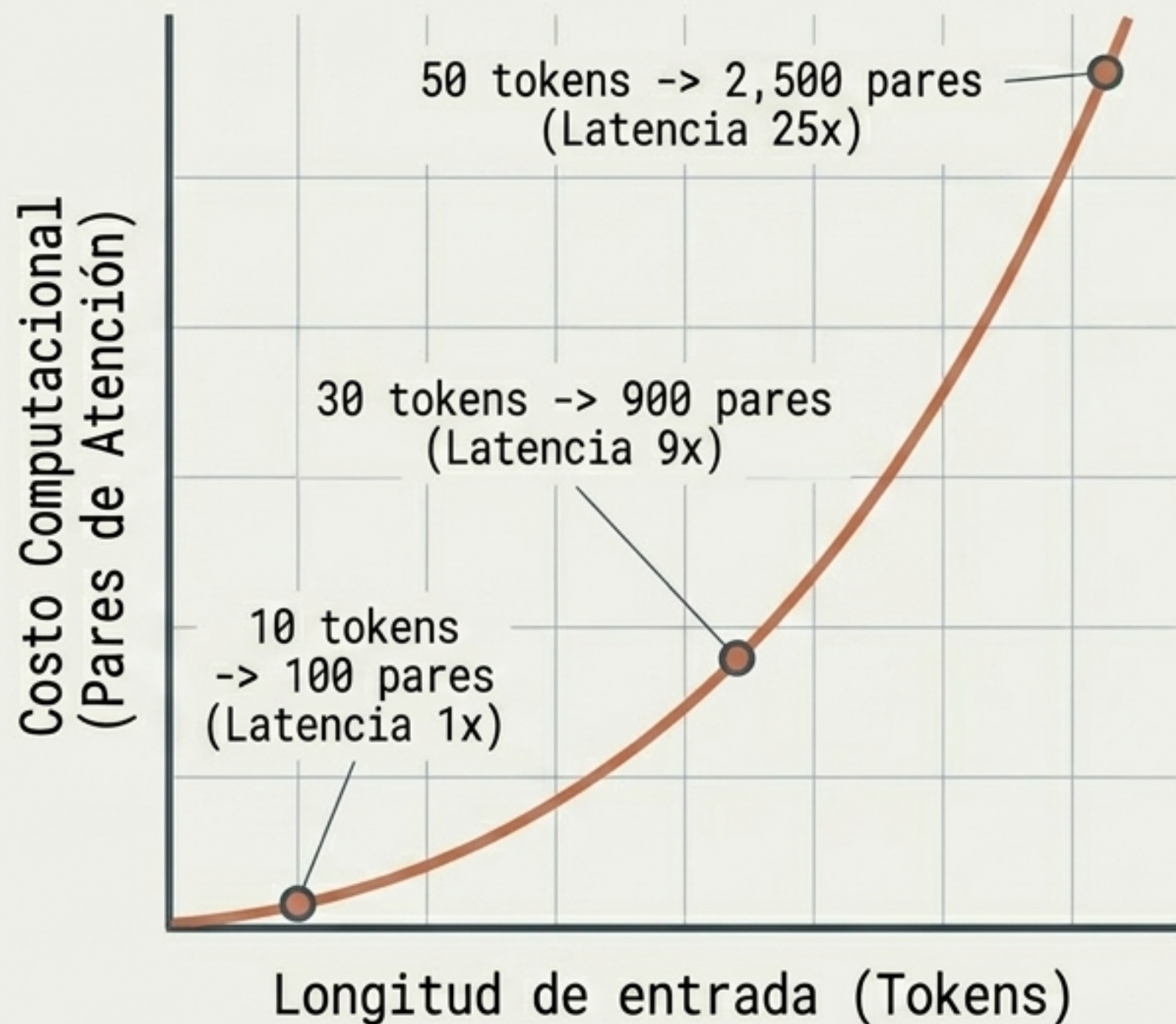
Mecánica LoRA

Low-Rank Adaptation aprovecha redundancias matemáticas. Modifica apenas entre el 0.1% y 1% del total de parámetros del sistema.

Impacto Operativo

Permite entrenar en un fin de semana con hardware estándar. Es posible intercambiar adaptadores al vuelo para distintos clientes sin duplicar el modelo base.

La física de la latencia: El costo cuadrático



Insight Crítico:

El self-attention calcula relaciones entre todos los tokens. Una secuencia con el doble de longitud no es el doble de lenta... es matemáticamente cuatro veces más lenta.

Procesar un simple documento de **45 tokens** exige resolver más de **2,000 relaciones** simultáneas en la capa de atención.

Tácticas de mitigación de latencia



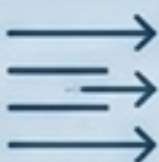
Control de Input

Restricciones de entrada estrictas y *tokenization* optimizada para el dominio (**menos fragmentación = menos tokens**).



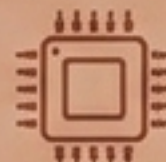
Selección de Modelo

Uso de modelos destilados o arquitecturas compactas y veloces adaptadas quirúrgicamente con LoRA.



Diseño de Infraestructura

Paralelización extrema y **flujos de enrutamiento asíncronos** para separar peticiones ligeras de documentos pesados.



Gestión de Memoria

Control de contexto dinámico y *caching* proactivo de *embeddings* frecuentes para **evitar recalcular la atención**.

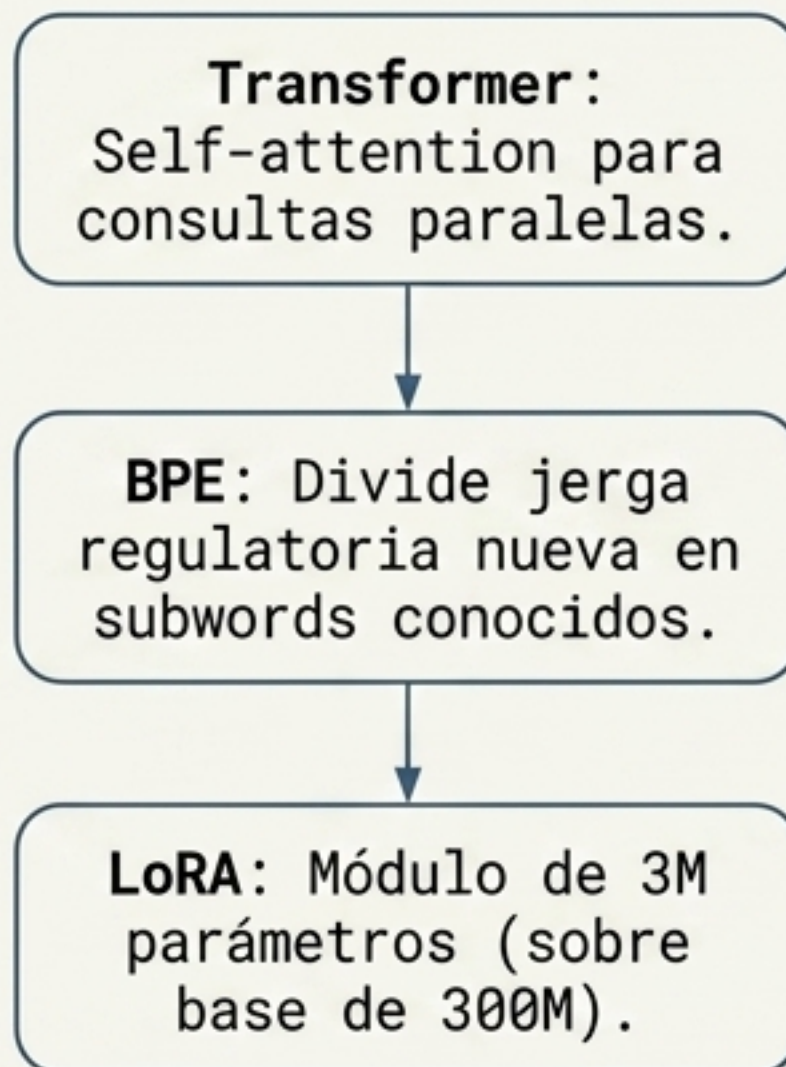
Caso de Estudio Integrador: Fintrust AI Labs

Misión: Adaptar un chatbot financiero a nuevas regulaciones de transparencia.

Restricciones Físicas

- Latencia estricta < 600ms
- Hardware GPU muy limitado
- Nueva jerga regulatoria 00V

Decisiones Arquitectónicas



Resultados

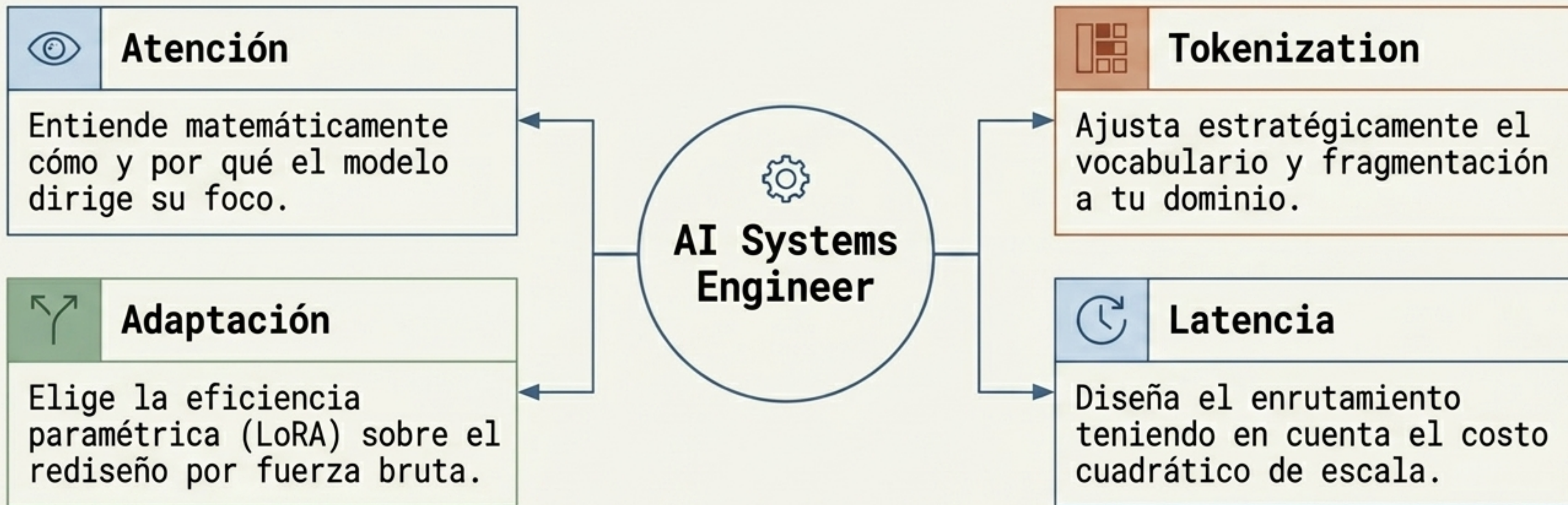
Adaptación completada en 4 horas.

Latencia promedio: 540 ms.

Costos de hardware: -70%.

De usar modelos a diseñar sistemas

Ya no alcanza con conectar una API de caja negra y esperar que funcione en el mundo real.



Un ingeniero moderno justifica cada componente. Entender los mecanismos internos es la diferencia entre un sistema que simplemente funciona y uno que escala sin fricción.